



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks

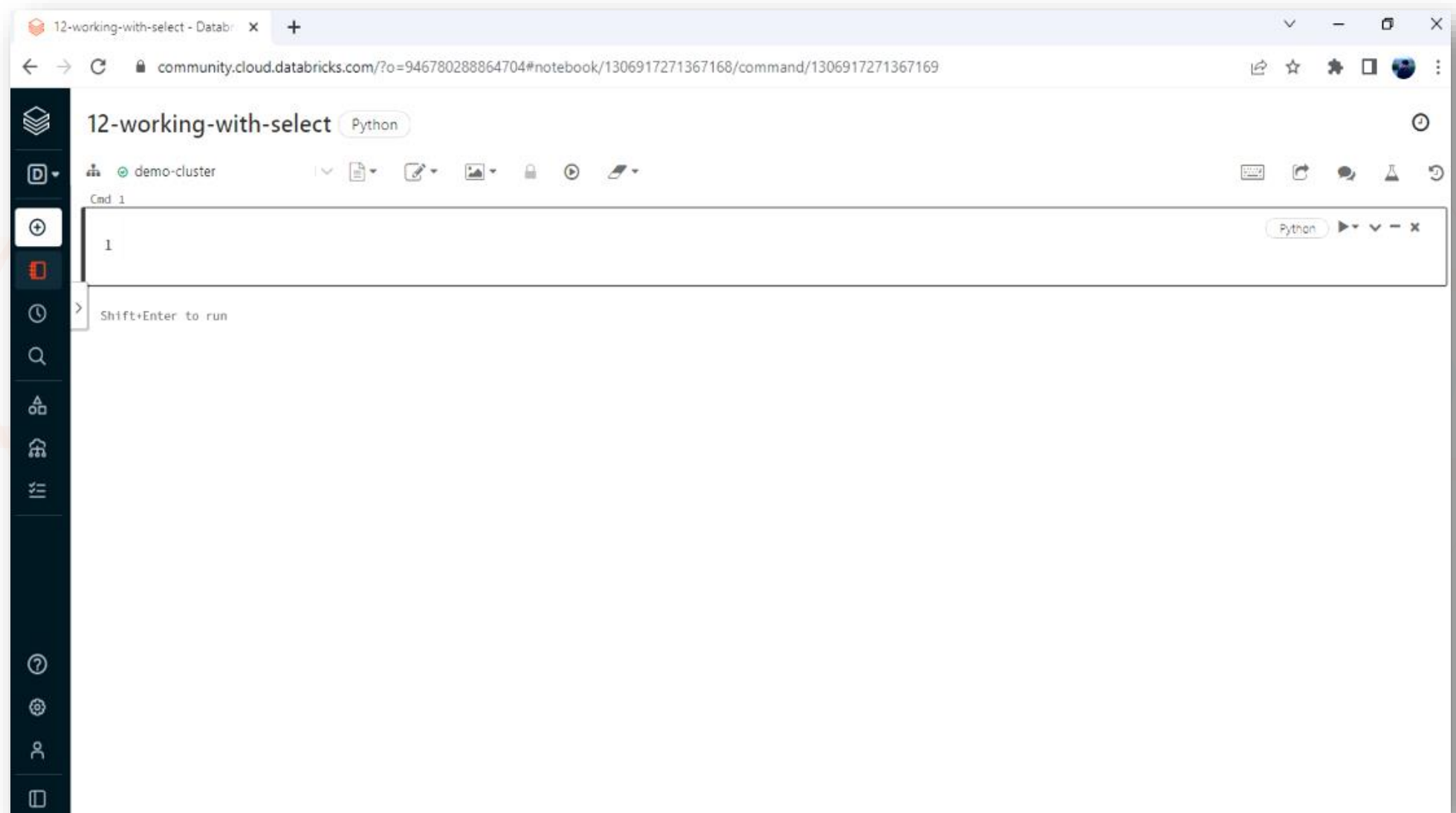


Working with Select and SelectExpr

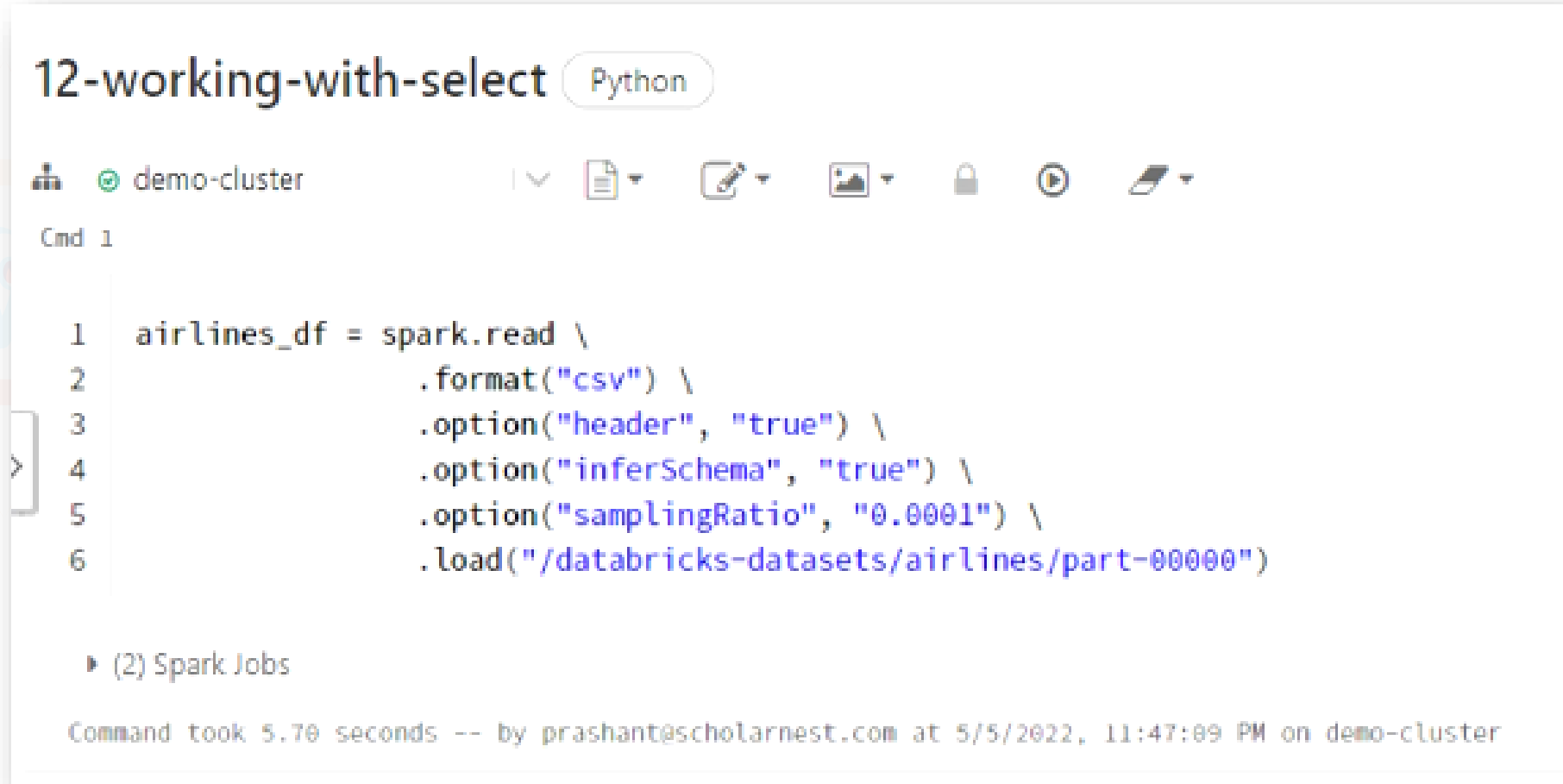
Spark gives you two methods to select one or more columns from your dataframe.

1. `select()`
2. `selectExpr()`

Go to your Databricks workspace and create a new notebook (**Reference - 12-working-with-select**)



We have created a Dataframe for using the select() method. I am reading one airline sample data from Databricks samples dataset and creating a Dataframe.



The screenshot shows a Databricks notebook interface. At the top, the notebook title is "12-working-with-select" and the language is set to "Python". Below the title, there's a toolbar with icons for cluster selection, file operations, and execution. The cluster name "demo-cluster" is visible. The code cell contains the following Python code:

```
1 airlines_df = spark.read \  
2     .format("csv") \  
3     .option("header", "true") \  
4     .option("inferSchema", "true") \  
5     .option("samplingRatio", "0.0001") \  
6     .load("/databricks-datasets/airlines/part-00000")
```

Below the code, it indicates "(2) Spark Jobs" and shows the execution status: "Command took 5.70 seconds -- by prashant@scholarnest.com at 5/5/2022, 11:47:09 PM on demo-cluster".

We have `airlines_df`, and we can start applying transformations. Let's assume this dataframe was a table. So you could run a `select *` query on this table like this - **SELECT * FROM table_name;**

12-working-with-select Python

demo-cluster Command took 5.76 seconds -- by prashant@scholarnest.com at 5/5/2022, 11:47:09 PM on demo-cluster

Cmd 2

```
1 display(airlines_df)
```

(1) Spark Jobs

	Year	Month	DayofMonth	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	TailNum
1	1987	10	14	3	741	730	912	849	PS	1451	NA
2	1987	10	15	4	729	730	903	849	PS	1451	NA
3	1987	10	17	6	741	730	918	849	PS	1451	NA
4	1987	10	18	7	729	730	847	849	PS	1451	NA
5	1987	10	19	1	749	730	922	849	PS	1451	NA
6	1987	10	21	3	728	730	848	849	PS	1451	NA
7	1987	10	22	4	728	730	852	849	PS	1451	NA

Truncated results: showing first 1000 rows.

Command took 0.93 seconds -- by prashant@scholarnest.com at 5/5/2022, 11:47:24 PM on demo-cluster

Cmd 3

So I am selecting four columns here in this code. The select method takes a comma-separated list of column names and returns only those columns. You can give column names or column objects. The select method is designed to accept both, and you can also mix the argument types. You can give a column name or supply a column object in the same select expression.

```
Cmd 4
1 from pyspark.sql.functions import col, column
2
3 airlines_df.select("CRSDepTime", col("DepTime"), airlines_df.CRSArrTime, column("ArrTime")).show(5)
```

► (1) Spark Jobs

CRSDepTime	DepTime	CRSArrTime	ArrTime
730	741	849	912
730	729	849	903
730	741	849	918
730	729	849	847
730	749	849	922

only showing top 5 rows

Command took 0.85 seconds -- by prashant@scholarnest.com at 5/5/2022, 11:51:17 PM on demo-cluster

You can also give expressions that result in a column object as shown in the code below. The first argument is a column name. The second argument is also a column name, but I am wrapping it inside the `expr()` function. The `expr()` function evaluates an expression and returns a column object. We do not have any expression here. `DepTime` is just a column name. So I should not have wrapped it inside the `expr()` function. However, the `expr()` function doesn't complain. It will simply evaluate the column name to a column object. The third argument is an expression. I am deducting the scheduled departure time from the departure time to find the delay and renaming the result as a delay.

```
Cmd 5
1  from pyspark.sql.functions import expr
2
3  airlines_df.select("CRSDepTime", expr("DepTime"), expr("DepTime - CRSDepTime as delay")).show(5)

▶ (1) Spark Jobs

+-----+-----+-----+
|CRSDepTime|DepTime|delay|
+-----+-----+-----+
|      730|      741|    11|
|      730|      729|    -1|
|      730|      741|    11|
|      730|      729|    -1|
|      730|      749|    19|
+-----+-----+-----+
only showing top 5 rows

Command took 0.71 seconds -- by prashant@scholarnest.com at 5/5/2022, 11:53:05 PM on demo-cluster
```

The select() method takes only two types of arguments.

1. Column Name
2. Column Object

You can see some negative delays. I don't want them. I mean, the flight went early. But we are calculating delay here. So I want to see only positive delays and set all the negative values to 0.
How can we do it?

```
Cmd 5

1 from pyspark.sql.functions import expr
2
3 airlines_df.select("CRSDepTime", expr("DepTime"), expr("DepTime - CRSDepTime as delay")).show(5)

▶ (1) Spark Jobs

+-----+-----+-----+
|CRSDepTime|DepTime|delay|
+-----+-----+-----+
|730|741|11|
|730|729|-1|
|730|741|11|
|730|729|-1|
|730|749|19|
+-----+-----+-----+

only showing top 5 rows

Command took 0.71 seconds -- by prashant@scholarnest.com at 5/5/2022, 11:53:05 PM on demo-cluster
```

Here is the code for that. I have the same SQL expression, and I evaluate it using the `expr()` function.

The `expr()` function will evaluate the expression to a column object, and it goes to the `select()` method.

```
> Cmd 6
1  airlines_df.select("CRSDepTime", "DepTime", expr("case when (DepTime - CRSDepTime) > 0 then (DepTime - CRSDepTime) else 0 end as delay")).show(5)

► (1) Spark Jobs

+-----+-----+-----+
|CRSDepTime|DepTime|delay|
+-----+-----+-----+
|      730|      741|    11|
|      730|      729|     0|
|      730|      741|    11|
|      730|      729|     0|
|      730|      749|    19|
+-----+-----+-----+
only showing top 5 rows

Command took 0.97 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:01:08 AM on demo-cluster
```

Sometime, using the `expr()` function inside the `select()` looks inconvenient.

I mean, using expressions in the `select()` method is a common thing.

We often use it. But we have a shortcut to avoid the `expr()` function explicitly, and that is using the `selectExpr()` method. The `selectExpr()` is the same as the `select`, but it takes an expression.

The `select` method takes a column name or column object. However, the `selectExpr()` only takes SQL-like expressions.

```
> Cmd 7  
1  airlines_df.selectExpr("CRSDepTime", "DepTime", "case when (DepTime - CRSDepTime) > 0 then (DepTime - CRSDepTime) else 0 end as delay").show(5)  
  
▶ (1) Spark Jobs  
  
+-----+-----+-----+  
|CRSDepTime|DepTime|delay|  
+-----+-----+-----+  
|      730|      741|    11|  
|      730|      729|     0|  
|      730|      741|    11|  
|      730|      729|     0|  
|      730|      749|    19|  
+-----+-----+-----+  
only showing top 5 rows  
  
Command took 0.49 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:02:16 AM on demo-cluster
```

Here is another example.

Here third argument to the select() method is using a dataframe expression.

And the select() method takes it, because the expression ultimately evaluates to a column object. But if you try using selectExpr(), you will see an error.

```
> Cmd 8
1  from pyspark.sql.functions import col
2
3  some_v = 15
4  airlines_df.select("CRSDepTime", "DepTime", col("DepTime") + some_v).show(5)
```

► (1) Spark Jobs

CRSDepTime	DepTime	(DepTime + 15)
730	741	756
730	729	744
730	741	756
730	729	744
730	749	764

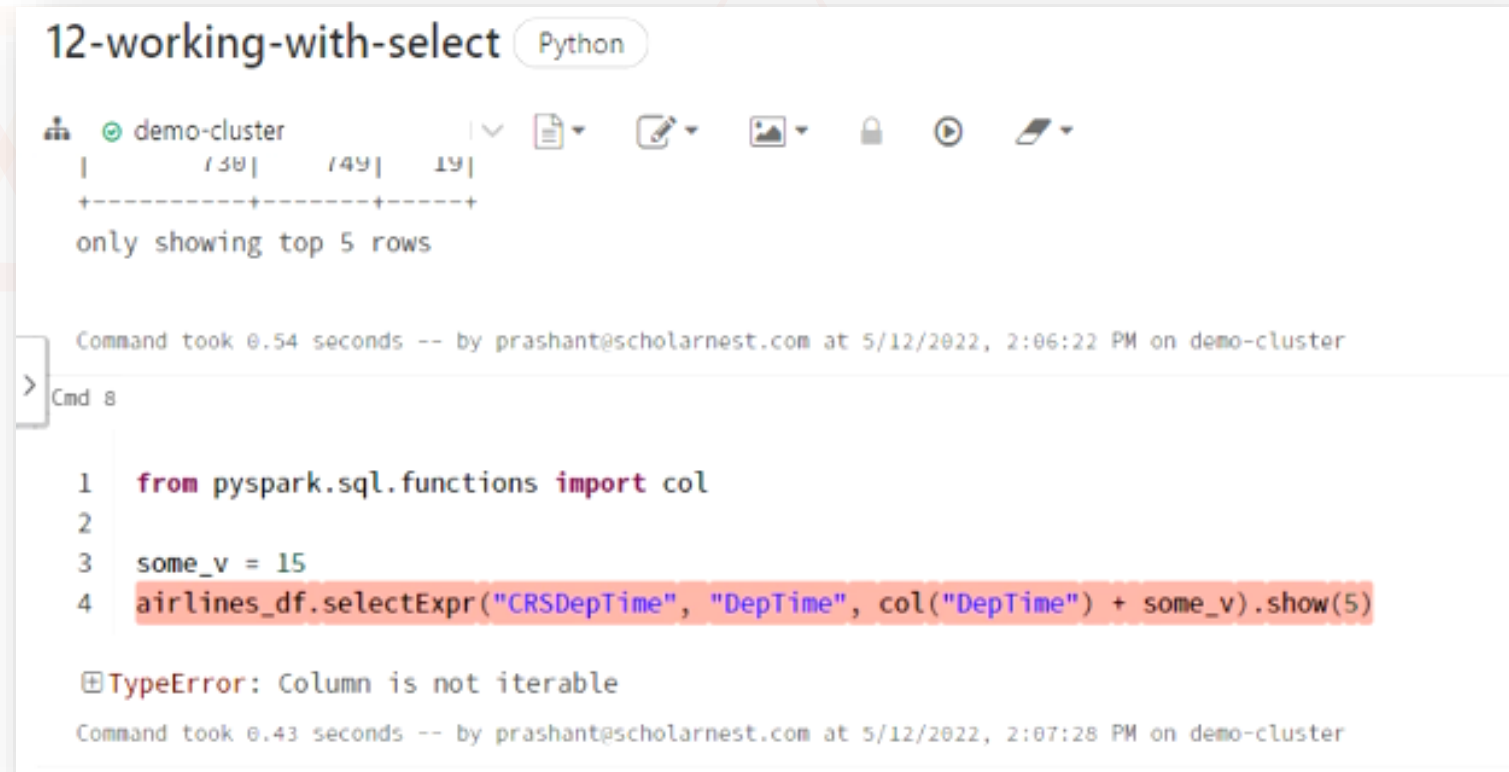
only showing top 5 rows

Command took 0.70 seconds -- by prashant@scholarnest.com at 5/6/2022, 12:04:58 AM on demo-cluster

Here we are trying to use the `selectExpr()`, and you can see an error.

So understand the difference between `select()` and `selectExpr()`. The `select` method takes a column name and a column object. You can use expressions as long as it evaluates a column object.

The `selectExpr()` only takes SQL-like expressions. You can also use column names because they evaluate as an expression.



The screenshot shows a Databricks workspace interface. At the top, the title is "12-working-with-select" with a "Python" language selector. Below the title, there's a toolbar with icons for cluster management, file operations, and other workspace functions. A cluster named "demo-cluster" is shown with a status of "Running" and a size of "138". Below the cluster information, there's a table with 3 columns and 19 rows, with a note "only showing top 5 rows". The main area displays a command execution history. The command executed was:

```
1 from pyspark.sql.functions import col
2
3 some_v = 15
4 airlines_df.selectExpr("CRSDepTime", "DepTime", col("DepTime") + some_v).show(5)
```

The command took 0.54 seconds and was executed by prashant@scholarnest.com at 5/12/2022, 2:06:22 PM on demo-cluster. Below the command, the error message is displayed:

```
⊞ TypeError: Column is not iterable
```

The command took 0.43 seconds and was executed by prashant@scholarnest.com at 5/12/2022, 2:07:28 PM on demo-cluster.



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com